BACKEND ENGINEERING

Complete Interview Preparation Guide

User Authentication System: Node.js | Express.js | Ruby on Rails

---

PART 1: The 10-Step Backend Project Implementation Framework

Every professional backend project follows this exact sequence. Memorize this like your ABCs — interviewers love seeing structured thinking.

01	**Understand Requirements & Define Scope** Read the problem carefully. Identify: entities, relationships, user roles, edge cases. Write a 5-line problem statement in your own words.
02	**Design the Data Model (ERD)** Draw your tables/collections on paper. Define fields, types, constraints, foreign keys. Normalise to 3NF or decide when to denormalize.
03	**Define Your API Contracts (REST/GraphQL)** List every endpoint: method, route, request body, response body, status codes. Document BEFORE coding.
04	**Setup Project & Folder Structure** Initialize project, install dependencies, configure environment (.env), set up linting and Git.
05	**Database Setup & Migrations** Create DB, write migration files for each table/model, seed test data.
06	**Implement Core Business Logic (Services Layer)** Write pure functions/classes that execute business rules. No HTTP here — only logic.

07	**Build Controllers / Route Handlers** Wire HTTP requests to service functions. Handle request parsing and response formatting.
08	**Add Middleware (Auth, Validation, Logging)** JWT verification, request validation schemas, rate limiting, error handling middleware.
09	**Write Tests (Unit + Integration)** Test services in isolation, test API endpoints end-to-end. Aim for 80%+ coverage on critical paths.
10	**Deploy & Monitor** Dockerize, set up CI/CD, configure environment variables in production, add health check endpoint.

PART 2: AUTH CONCEPTS YOU MUST KNOW

What is JWT?

JSON Web Token = 3 base64-encoded parts joined by dots:

Header. Payload. Signature

Header: algorithm (HS256). Payload: userId, exp, iat. Signature: HMAC of header+payload using secret.

Stateless — server does NOT store sessions. Just verify signature.

Access vs Refresh Tokens

Access Token: Short-lived (15min). Sent with every request.

Refresh Token: Long-lived (7-30 days). Stored in HttpOnly cookie. Used ONLY to get new access tokens.

Why two? Security: if access token leaks, attacker has max 15 min.

bcrypt Password Hashing

NEVER store plain text passwords. bcrypt applies:

1. Salt: random bytes added to password before hashing

2. Cost factor (10-12): controls iterations (2^cost)

3. One-way: cannot reverse the hash

salt_rounds = 12 is production standard

Security Best Practices

HttpOnly Cookie: JS cannot read it (XSS protection)

CORS configuration: whitelist specific origins

Rate limiting: max 5 login attempts / 15 minutes

HTTPS only in production (never HTTP for auth)

Env vars for ALL secrets — never hardcode

PART 3: PURE NODE.JS — AUTH FROM SCRATCH (NO FRAMEWORK)

This is the foundation. Understanding this makes Express trivial. Interviewers use this to test if you truly understand HTTP.

Step 1 — Project Setup

```
# Terminal commands:
mkdir node-auth && cd node-auth
npm init -y
npm install jsonwebtoken bcryptjs

# Folder structure:
node-auth/
  server.js          <- entry point
  routes/auth.js     <- login/logout/register handlers
  middleware/auth.js <- JWT verification
  db.js              <- in-memory users store (for demo)
  .env               <- JWT_SECRET, PORT
```

Step 2 — In-Memory Database (db.js)

In interviews, use an in-memory array for DB to save setup time. Tell the interviewer "in production I would use PostgreSQL/MongoDB."

```
// db.js
// Simulates a database using a plain object
// In production: replace with Prisma/Mongoose queries

const users = [];  // { id, email, passwordHash }
let nextId = 1;

module.exports = {
```

```
  findByEmail: (email) => users.find(u => u.email === email),
  findById:    (id)    => users.find(u => u.id === id),
  create:      (user)  => { user.id = nextId++; users.push(user); return user; }
};
```

Step 3 — Auth Route Handlers (routes/auth.js)

```
// routes/auth.js
const bcrypt = require('bcryptjs');
const jwt    = require('jsonwebtoken');
const db     = require('../db');

const JWT_SECRET      = process.env.JWT_SECRET || "dev-secret-CHANGE-IN-PROD";
const ACCESS_EXPIRES  = "15m";
const REFRESH_EXPIRES = "7d";

// ──────── REGISTER ──────────────────────────────────────────────────
async function register(req, res) {
  try {
    const { email, password } = parseBody(req);

    if (!email || !password)
      return sendJSON(res, 400, { error: "email and password required" });

    if (db.findByEmail(email))
      return sendJSON(res, 409, { error: "email already registered" });

    const passwordHash = await bcrypt.hash(password, 12); // cost=12
    const user = db.create({ email, passwordHash });

    sendJSON(res, 201, { message: "Registered", userId: user.id });
  } catch (err) {
    sendJSON(res, 500, { error: "Internal error" });
  }
}

// ──────── LOGIN ─────────────────────────────────────────────────────
async function login(req, res) {
  try {
    const { email, password } = parseBody(req);

    const user = db.findByEmail(email);
    if (!user) return sendJSON(res, 401, { error: "Invalid credentials" });

    const valid = await bcrypt.compare(password, user.passwordHash);
    if (!valid) return sendJSON(res, 401, { error: "Invalid credentials" });

    // Generate tokens
    const accessToken  = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn:
ACCESS_EXPIRES });
    const refreshToken = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn:
REFRESH_EXPIRES });

    // Send refresh in HttpOnly cookie (XSS-safe)
    res.setHeader('Set-Cookie', [
      `refreshToken=${refreshToken}; HttpOnly; Path=/; Max-Age=${7*24*3600};
SameSite=Strict`
    ]);

    sendJSON(res, 200, { accessToken, message: "Login successful" });
  } catch (err) {
```

```
    sendJSON(res, 500, { error: "Internal error" });
  }
}

// ──────── LOGOUT ──────────────────────────────────────────────────
function logout(req, res) {
  // Clear the HttpOnly cookie by setting Max-Age=0
  res.setHeader('Set-Cookie', 'refreshToken=; HttpOnly; Path=/; Max-Age=0;
SameSite=Strict');
  sendJSON(res, 200, { message: "Logged out" });
}

// ──────── GET CURRENT USER (protected) ────────────────────────────
function me(req, res) {
  // req.userId set by auth middleware
  const user = db.findById(req.userId);
  if (!user) return sendJSON(res, 404, { error: "User not found" });
  sendJSON(res, 200, { id: user.id, email: user.email });
}

// ──────── HELPERS ─────────────────────────────────────────────────
function parseBody(req) {
  // Body was pre-parsed and attached by server.js
  return req.body || {};
}

function sendJSON(res, status, data) {
  res.writeHead(status, { "Content-Type": "application/json" });
  res.end(JSON.stringify(data));
}

module.exports = { register, login, logout, me };
```

Step 4 — Auth Middleware (middleware/auth.js)

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

const JWT_SECRET = process.env.JWT_SECRET || "dev-secret-CHANGE-IN-PROD";

function authenticate(req, res, next) {
  // Expect: Authorization: Bearer <token>
  const authHeader = req.headers['authorization'];

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    res.writeHead(401, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ error: "No token provided" }));
    return;
  }

  const token = authHeader.slice(7); // Remove "Bearer "

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.userId = decoded.userId; // Attach userId to request
    next();                      // Proceed to route handler
  } catch (err) {
    res.writeHead(401, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ error: "Invalid or expired token" }));
  }
}
```

```
module.exports = { authenticate };
```

Step 5 — Main Server (server.js)

```
// server.js  — Pure Node.js HTTP server (zero frameworks)
const http   = require('http');
const url    = require('url');
const auth   = require('./routes/auth');
const { authenticate } = require('./middleware/auth');

require("dotenv").config();

const PORT = process.env.PORT || 3000;

// Helper: parse JSON body from request stream
function parseBodyMiddleware(req, res, next) {
  let raw = "";
  req.on('data', chunk => { raw += chunk.toString(); });
  req.on('end', () => {
    try { req.body = JSON.parse(raw || "{}"); } catch { req.body = {}; }
    next();
  });
}

const server = http.createServer((req, res) => {
  const { pathname } = url.parse(req.url);

  // Route the request
  parseBodyMiddleware(req, res, () => {
    if (pathname === '/register' && req.method === 'POST')  return auth.register(req,
res);
    if (pathname === '/login'    && req.method === 'POST')  return auth.login(req, res);
    if (pathname === '/logout'   && req.method === 'POST')  return auth.logout(req, res);

    // Protected route — run auth middleware first
    if (pathname === '/me' && req.method === 'GET') {
      return authenticate(req, res, () => auth.me(req, res));
    }

    // 404 fallback
    res.writeHead(404, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ error: "Route not found" }));
  });
});

server.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
```

Interview Tip — Pure Node.js

Key points to mention: (1) Node HTTP creates a request/response duplex stream. (2) You must manually parse the request body from data events. (3) Routing is manual — this is exactly what Express abstracts. (4) jwt.verify() throws on invalid/expired tokens — always wrap in try/catch.

PART 4: EXPRESS.JS AUTH SYSTEM

Express adds: automatic body parsing, clean middleware chaining with next(), and a router. This is what 90% of Node.js jobs use.

Project Structure

```
express-auth/
  src/
    config/          db.js, env.js
    controllers/     auth.controller.js
    middleware/      auth.middleware.js, validate.middleware.js
    routes/          auth.routes.js
    services/        auth.service.js    <- BUSINESS LOGIC (key pattern!)
    app.js           <- Express setup
    server.js        <- HTTP server start
  .env
  package.json
```

Setup & Dependencies

```
npm init -y
npm install express bcryptjs jsonwebtoken cookie-parser express-rate-limit
npm install dotenv
npm install --save-dev nodemon

# package.json scripts:
"scripts": {
  "start": "node src/server.js",
  "dev":   "nodemon src/server.js"
}
```

The Service Layer — auth.service.js (Most Important!)

CRITICAL PATTERN: Business logic lives in services, NOT controllers. Controllers only handle HTTP. This is what separates juniors from seniors.

```
// src/services/auth.service.js
const bcrypt = require('bcryptjs');
const jwt    = require('jsonwebtoken');

const users  = [];  // Replace with DB queries in production
let   nextId = 1;

const SALT_ROUNDS     = 12;
const ACCESS_EXPIRES  = "15m";
const REFRESH_EXPIRES = "7d";

// ─── Register ──────────────────────────────────────────────────
async function registerUser({ email, password }) {
  const exists = users.find(u => u.email === email);
  if (exists) throw new Error("EMAIL_EXISTS");

  const passwordHash = await bcrypt.hash(password, SALT_ROUNDS);
  const user = { id: nextId++, email, passwordHash };
  users.push(user);
```

```
  return { id: user.id, email: user.email };  // Never return passwordHash!
}

// ─── Login ──────────────────────────────────────────────────────
async function loginUser({ email, password }) {
  const user = users.find(u => u.email === email);
  if (!user) throw new Error("INVALID_CREDENTIALS");

  const valid = await bcrypt.compare(password, user.passwordHash);
  if (!valid) throw new Error("INVALID_CREDENTIALS");  // Same error — no info leak!

  const payload = { userId: user.id };
  const secret  = process.env.JWT_SECRET;

  const accessToken  = jwt.sign(payload, secret, { expiresIn: ACCESS_EXPIRES });
  const refreshToken = jwt.sign(payload, secret, { expiresIn: REFRESH_EXPIRES });

  return { accessToken, refreshToken };
}

// ─── Get User by ID ─────────────────────────────────────────────
async function getUserById(id) {
  const user = users.find(u => u.id === id);
  if (!user) throw new Error("USER_NOT_FOUND");
  return { id: user.id, email: user.email };
}

module.exports = { registerUser, loginUser, getUserById };
```

Controller — auth.controller.js

```
// src/controllers/auth.controller.js
const authService = require('../services/auth.service');

// ─── Register ───────────────────────────────────────────────────
async function register(req, res, next) {
  try {
    const user = await authService.registerUser(req.body);
    res.status(201).json({ message: "Registered successfully", user });
  } catch (err) {
    if (err.message === 'EMAIL_EXISTS')
      return res.status(409).json({ error: "Email already in use" });
    next(err);  // Pass unexpected errors to error middleware
  }
}

// ─── Login ──────────────────────────────────────────────────────
async function login(req, res, next) {
  try {
    const { accessToken, refreshToken } = await authService.loginUser(req.body);

    // Set refresh token in HttpOnly cookie
    res.cookie("refreshToken", refreshToken, {
      httpOnly: true,   // Not accessible by JavaScript
      secure: process.env.NODE_ENV === "production",  // HTTPS only in prod
      sameSite: 'strict',
      maxAge: 7 * 24 * 60 * 60 * 1000  // 7 days in ms
    });

    res.json({ accessToken });
```

```
  } catch (err) {
    if (err.message === 'INVALID_CREDENTIALS')
      return res.status(401).json({ error: "Invalid email or password" });
    next(err);
  }
}

// ─── Logout ──────────────────────────────────────────────────────
function logout(req, res) {
  res.clearCookie("refreshToken", { httpOnly: true, sameSite: "strict" });
  res.json({ message: "Logged out successfully" });
}

// ─── Me (protected) ──────────────────────────────────────────────
async function me(req, res, next) {
  try {
    const user = await authService.getUserById(req.userId);
    res.json({ user });
  } catch (err) {
    next(err);
  }
}

module.exports = { register, login, logout, me };
```

Auth Middleware — middleware/auth.middleware.js

```
// src/middleware/auth.middleware.js
const jwt = require('jsonwebtoken');

function authenticate(req, res, next) {
  const authHeader = req.headers['authorization'];

  if (!authHeader?.startsWith("Bearer "))
    return res.status(401).json({ error: "Authorization header missing" });

  const token = authHeader.slice(7);

  try {
    const { userId } = jwt.verify(token, process.env.JWT_SECRET);
    req.userId = userId;  // Attach to request for downstream handlers
    next();
  } catch (err) {
    const message = err.name === "TokenExpiredError"
      ? "Token expired"
      : "Invalid token";
    res.status(401).json({ error: message });
  }
}

module.exports = { authenticate };
```

Routes — auth.routes.js

```
// src/routes/auth.routes.js
const router     = require('express').Router();
const controller = require('../controllers/auth.controller');
const { authenticate } = require('../middleware/auth.middleware');
```

```
// Public routes
router.post('/register', controller.register);
router.post('/login',    controller.login);
router.post('/logout',   controller.logout);

// Protected route — authenticate middleware runs first
router.get('/me', authenticate, controller.me);

module.exports = router;
```

Express App Setup — app.js

```
// src/app.js
const express    = require('express');
const cookieParser = require('cookie-parser');
const rateLimit  = require('express-rate-limit');
const authRoutes = require('./routes/auth.routes');

const app = express();

// ─── Core Middleware ──────────────────────────────────────────
app.use(express.json());           // Parse JSON bodies
app.use(cookieParser());           // Parse cookies

// ─── Rate Limiting ────────────────────────────────────────────
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 5,                     // 5 attempts per window
  message: { error: "Too many login attempts. Try again in 15 minutes." },
  standardHeaders: true
});
app.use('/login', loginLimiter);

// ─── Routes ───────────────────────────────────────────────────
app.use('/api/auth', authRoutes);   // All routes: /api/auth/register etc

// ─── Global Error Handler ─────────────────────────────────────
// MUST have 4 params: (err, req, res, next)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: "Something went wrong" });
});

module.exports = app;
```

Server Entry Point — server.js

```
// src/server.js
require("dotenv").config();
const app  = require('./app');

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Express auth server running on http://localhost:${PORT}`);
});
```

.env File

```
# .env  — NEVER commit this to Git!
PORT=3000
NODE_ENV=development
JWT_SECRET=replace-this-with-a-long-random-string-in-production
```

How to Test — curl Commands

```
# Register
curl -X POST http://localhost:3000/api/auth/register \
     -H "Content-Type: application/json" \
     -d '{"email":"alice@example.com","password":"SecurePass123"}'

# Login (returns accessToken)
curl -X POST http://localhost:3000/api/auth/login \
     -H "Content-Type: application/json" \
     -d '{"email":"alice@example.com","password":"SecurePass123"}'

# Me (replace TOKEN with value from login response)
curl http://localhost:3000/api/auth/me \
     -H "Authorization: Bearer TOKEN"

# Logout
curl -X POST http://localhost:3000/api/auth/logout
```

PART 5: RUBY ON RAILS — API AUTH SYSTEM

Rails follows Convention over Configuration. The same auth logic, but Rails gives you more structure out of the box. Learn the Rails way: fat models, skinny controllers.

Step 1 — Create API-Only Rails App

```
# Create a new Rails app in API mode (no views needed for auth API)
rails new rails-auth --api --database=sqlite3
cd rails-auth

# Add gems to Gemfile:
gem 'bcrypt',              '~> 3.1.7'   # Already in Gemfile, uncomment it
gem 'jwt',                 '~> 2.7'
gem 'rack-cors'                         # Cross-origin requests

# Then install:
bundle install
```

Step 2 — Generate User Model with Migration

```
# Generate the User model
# password_digest is the Rails convention for bcrypt
rails generate model User email:string:uniq password_digest:string
```

```
# Run migration (creates the users table)
rails db:migrate

# The migration file created (db/migrate/xxxxx_create_users.rb):
class CreateUsers < ActiveRecord::Migration[7.0]
  def change
    create_table :users do |t|
      t.string :email,           null: false
      t.string :password_digest, null: false
      t.timestamps
    end
    add_index :users, :email, unique: true
  end
end
```

Step 3 — User Model (app/models/user.rb)

```
# app/models/user.rb
class User < ApplicationRecord
  # has_secure_password: adds password= setter with bcrypt hashing,
  # adds authenticate(password) method, validates password presence
  has_secure_password

  # Validations
  validates :email, presence: true,
                    uniqueness: { case_sensitive: false },
                    format: { with: URI::MailTo::EMAIL_REGEXP }

  validates :password, length: { minimum: 8 }, allow_nil: true

  # Normalize email before saving
  before_save { self.email = email.downcase.strip }
end

# WHAT has_secure_password DOES FOR YOU:
# user.password = "secret"      => hashes with bcrypt and stores in password_digest
# user.authenticate("secret")   => returns user if match, false if not
# user.authenticate("wrong")    => returns false (safe, no exception)
```

Step 4 — JWT Service (app/services/jwt_service.rb)

```
# app/services/jwt_service.rb
# Plain Ruby class — no Rails magic, just JWT encoding/decoding
class JwtService
  SECRET       = Rails.application.credentials.jwt_secret || ENV["JWT_SECRET"]
  ALGORITHM    = "HS256"
  ACCESS_TTL   = 15.minutes
  REFRESH_TTL  = 7.days

  def self.encode(payload, expiry = ACCESS_TTL)
    payload = payload.merge(exp: expiry.from_now.to_i)
    JWT.encode(payload, SECRET, ALGORITHM)
  end

  def self.decode(token)
    decoded = JWT.decode(token, SECRET, true, { algorithm: ALGORITHM })
    HashWithIndifferentAccess.new(decoded.first)  # Access as hash
  rescue JWT::ExpiredSignature
    raise "TOKEN_EXPIRED"
  rescue JWT::DecodeError
```

```
    raise "TOKEN_INVALID"
  end
end
```

Step 5 — Auth Controller (app/controllers/api/v1/auth_controller.rb)

```
# Create the directory: mkdir -p app/controllers/api/v1

# app/controllers/api/v1/auth_controller.rb
module Api
  module V1
    class AuthController < ApplicationController
      before_action :authenticate_user!, only: [:me]

      # POST /api/v1/auth/register
      def register
        user = User.new(user_params)

        if user.save
          render json: { message: "Registered", user: { id: user.id, email: user.email }
},
                 status: :created
        else
          render json: { errors: user.errors.full_messages }, status:
:unprocessable_entity
        end
      end

      # POST /api/v1/auth/login
      def login
        user = User.find_by(email: params[:email]&.downcase&.strip)

        # authenticate() comes from has_secure_password
        if user&.authenticate(params[:password])
          access_token  = JwtService.encode({ user_id: user.id })
          refresh_token = JwtService.encode({ user_id: user.id },
JwtService::REFRESH_TTL)

          # Set refresh token as HttpOnly cookie
          cookies.encrypted[:refresh_token] = {
            value:     refresh_token,
            http_only: true,
            secure:    Rails.env.production?,
            same_site: :strict,
            expires:   7.days.from_now
          }

          render json: { access_token: access_token }
        else
          render json: { error: "Invalid email or password" }, status: :unauthorized
        end
      end

      # DELETE /api/v1/auth/logout
      def logout
        cookies.delete(:refresh_token)
        render json: { message: "Logged out" }
      end

      # GET /api/v1/auth/me  (protected)
      def me
        render json: { user: { id: @current_user.id, email: @current_user.email } }
```

```
      end

      private

      def user_params
        params.permit(:email, :password, :password_confirmation)
      end
    end
  end
end
```

Step 6 — Application Controller with Auth

```
# app/controllers/application_controller.rb
class ApplicationController < ActionController::API
  # Makes @current_user available in any controller
  attr_reader :current_user

  private

  def authenticate_user!
    header = request.headers["Authorization"]

    unless header&.start_with?("Bearer ")
      return render json: { error: "Authorization header missing" }, status:
:unauthorized
    end

    token = header.split(" ").last

    begin
      payload = JwtService.decode(token)
      @current_user = User.find(payload[:user_id])
    rescue => e
      render json: { error: e.message }, status: :unauthorized
    end
  end
end
```

Step 7 — Routes (config/routes.rb)

```
# config/routes.rb
Rails.application.routes.draw do
  namespace :api do
    namespace :v1 do
      scope :auth do
        post   'register', to: 'auth#register'
        post   'login',    to: 'auth#login'
        delete 'logout',   to: 'auth#logout'
        get    'me',       to: 'auth#me'
      end
    end
  end
end

# Routes generated:
# POST   /api/v1/auth/register
# POST   /api/v1/auth/login
# DELETE /api/v1/auth/logout
# GET    /api/v1/auth/me
```

Step 8 — CORS Configuration (config/initializers/cors.rb)

```
# config/initializers/cors.rb
Rails.application.config.middleware.insert_before 0, Rack::Cors do
  allow do
    origins "http://localhost:3001"  # Your frontend URL

    resource "*",
      headers:     :any,
      methods:     [:get, :post, :put, :patch, :delete, :options, :head],
      credentials: true  # Required for cookies
  end
end
```

Step 9 — Test with Rails Console

```
# Start server: rails server -p 3000

# Test in Rails console: rails console
user = User.create!(email: "test@example.com", password: "password123")
user.authenticate("password123")  # => returns user object (truthy)
user.authenticate("wrongpass")    # => false

# Encode a JWT manually in console:
JwtService.encode({ user_id: user.id })

# Test register endpoint:
curl -X POST http://localhost:3000/api/v1/auth/register \
     -H "Content-Type: application/json" \
     -d
'{"email":"alice@example.com","password":"SecurePass123","password_confirmation":"SecureP
ass123"}'
```

PART 6: SIDE-BY-SIDE COMPARISON

Concept	Pure Node.js	Express.js	Ruby on Rails
Password hashing	bcrypt.hash() manually	bcrypt.hash() in service	has_secure_password (built-in)
Routing	Manual if/else on URL	Express Router	routes.rb + namespaces
Middleware	Manual function chain	app.use() / next()	before_action filters
Body parsing	req.on("data") stream	express.json()	Automatic (ActionController)

Concept	Pure Node.js	Express.js	Ruby on Rails
JWT	jsonwebtoken manually	jsonwebtoken in service	jwt gem in service class
DB layer	In-memory array	Sequelize/Prisma/Mongoose	ActiveRecord ORM
Error handling	try/catch + res.end()	Error middleware (4 params)	rescue_from in controller
Setup time	Fast (zero deps)	Medium	Longer (convention-heavy)
Best for	Learning HTTP deeply	APIs, microservices	Rapid full-stack apps

---

PART 7: TOP INTERVIEW QUESTIONS & ANSWERS

Q: Why HttpOnly cookie for refresh token, not localStorage?

Because localStorage is accessible by JavaScript. If your app has any XSS vulnerability, an attacker can steal tokens with document.cookie or localStorage.getItem(). HttpOnly cookies cannot be read by JavaScript at all — they are only sent automatically by the browser on matching requests. This is non-negotiable in production auth systems.

Q: Why do we not store JWT on the server?

That is the whole point of JWT — it is stateless. The server issues a signed token. On every subsequent request, the server just verifies the signature using its secret key. No database lookup needed. This is what makes JWT horizontally scalable (works across multiple server instances without a shared session store).

Q: How do you invalidate a JWT before expiry (logout)?

Strictly speaking, you cannot invalidate a stateless JWT. Solutions: (1) Short expiry (15 min) — worst case attacker has 15 min. (2) Refresh token rotation — store refresh tokens in DB, delete on logout. (3) Token blacklist — Redis set of invalidated tokens, checked on each request. Option 3 is most common for security-critical apps. For the access token, clients should delete it from memory on logout.

Q: What is the difference between authentication and authorization?

Authentication = "Who are you?" — Verifying identity. Logging in with email/password and receiving a JWT is authentication.

Authorization = "What are you allowed to do?" — Checking permissions. Checking that a user has the "admin" role before letting them delete a record is authorization. JWT can carry role claims in the payload for authorization.

Q: Why use the same error message "Invalid credentials" for both wrong email AND wrong password?

Security: If you return "Email not found" for wrong email and "Wrong password" for wrong password, attackers can use your login form to enumerate valid user emails. By returning the same error for both cases, you give away no information about whether the email exists in your database.

PART 8: THE MENTAL MODEL — DRAW THIS IN ANY INTERVIEW

When asked to design an auth system, draw this flow on the whiteboard:

```
CLIENT                          SERVER
  |                               |
  |--- POST /register ------------>|
  |    { email, password }         |  1. Validate input
  |                               |  2. bcrypt.hash(password, 12)
  |<-- 201 Created ----------------+  3. Save { email, passwordHash } to DB
  |                               |
  |--- POST /login --------------->|
  |    { email, password }         |  1. Find user by email
  |                               |  2. bcrypt.compare(password, hash)
  |                               |  3. jwt.sign({ userId }, secret, 15m)
  |<-- 200 { accessToken } --------+  4. Set refreshToken in HttpOnly cookie
  |    + Set-Cookie: refreshToken  |
  |                               |
  |--- GET /me (protected) ------->|
  |    Authorization: Bearer TOKEN  |  1. Extract token from header
  |                               |  2. jwt.verify(token, secret)
  |<-- 200 { user } ---------------+  3. Find user by decoded userId
  |                               |
  |--- POST /logout ------------->|
  |                               |  1. Clear refreshToken cookie
  |<-- 200 { message } -----------+  2. Client discards accessToken from memory
```

FINAL SENIOR ENGINEER TIPS

DO in interviews	AVOID in interviews
Think aloud — say what you are doing	Storing passwords in plain text
Mention production concerns (HTTPS, env vars, rate limiting)	Hardcoding secrets (JWT_SECRET = "abc123")
Use a service layer to separate concerns	Storing tokens in localStorage without mentioning XSS risk
Say "in-memory for now, in production I would use PostgreSQL"	Using short salt rounds (bcrypt < 10 is weak)
Explain your error message choices (security)	Forgetting try/catch around jwt.verify()

You now have everything a Senior Backend Engineer knows about Auth Systems. Practice building each implementation from memory.